

Entity Component System (ECS)

Architecture & Implementation Sparse Set

Adam HAMOUC

May 2026

1. Architecture globale

Trois acteurs distincts, chacun avec une responsabilite unique :

- **Entity** : un simple ID numerique. Pas de donnees, pas de logique. C'est une cle d'accès.
- **Component** : donnees pures, zero logique. **TransformComponent** contient position/rotation/scale, rien d'autre.
- **System** : logique pure, zero donnees propres. Itere sur les entites qui possedent les composants dont il a besoin et les traite en batch.

```
// Entity = juste un ID (index + generation pour detecter les entites mortes)
struct EntityID {
    uint32_t index;
    uint32_t generation; // invalide les vieux IDs apres destruction
    bool operator==(const EntityID&) const = default;
};

// Components = donnees pures
struct TransformComponent { glm::vec3 position; glm::quat rotation; glm::vec3 scale; };
struct VelocityComponent { glm::vec3 linear; glm::vec3 angular; };
struct HealthComponent { float current; float max; };
struct RenderComponent { MeshHandle mesh; MaterialHandle material; };
```

2. ComponentStorage — Sparse Set

Un storage dense par type de composant. Deux structures complementaires :

- **dense** : tableau compact des composants et des IDs (cache-friendly pour l'iteration).
- **sparse** : tableau indexe par EntityID.index, contient l'index dans dense (lookup O(1)).

```
template<typename T>
class ComponentStorage {
public:
    void add(EntityID id, T&& comp) {
        sparse[id.index] = dense.size(); // sparse pointe vers dense
        dense.push_back(std::forward<T>(comp));
        denseIDs.push_back(id);
    }

    void remove(EntityID id) {
        size_t idx = sparse[id.index];
        size_t last = dense.size() - 1;
        // Swap-and-pop : tableau reste dense
        std::swap(dense[idx], dense[last]);
        std::swap(denseIDs[idx], denseIDs[last]);
        sparse[denseIDs[idx].index] = idx; // mettre a jour le sparse
        sparse[id.index] = INVALID;
        dense.pop_back();
        denseIDs.pop_back();
    }

    bool has(EntityID id) const {
        return id.index < sparse.size() && sparse[id.index] != INVALID;
    }

    T& get(EntityID id) { return dense[sparse[id.index]]; }

    // Iteration batch : acces direct au tableau dense, cache-friendly
    T* data() { return dense.data(); }
    size_t size() { return dense.size(); }
    const std::vector<EntityID>& ids() { return denseIDs; }
```

```
private:
    static constexpr size_t INVALID = SIZE_MAX;
    std::vector<T> dense; // composants contigus en memoire
    std::vector<EntityID> denseIDs; // IDs correspondants dans dense
    std::vector<size_t> sparse; // sparse[entity.index] = index dans dense
};
```

3. EntityManager — generation d'IDs

La **generation** resout le probleme des IDs recycles : si une entite est detruite et son index reutilise, les vieux IDs sont invalides car leur generation ne correspond plus.

```
class EntityManager {
public:
    EntityID create() {
        if (!freeList.empty()) {
            uint32_t idx = freeList.back(); freeList.pop_back();
            return {idx, generations[idx]}; // nouvelle generation, meme index
        }
        generations.push_back(0);
        return {(uint32_t)generations.size() - 1, 0};
    }

    void destroy(EntityID id) {
        generations[id.index]++; // invalide tous les vieux IDs avec cet index
        freeList.push_back(id.index);
    }

    bool isAlive(EntityID id) const {
        return id.index < generations.size()
            && generations[id.index] == id.generation;
    }

private:
    std::vector<uint32_t> generations;
    std::vector<uint32_t> freeList; // index recycleables
};
```

4. World — interface unifiee

```
class World {
public:
    EntityID createEntity() { return entities.create(); }
    void destroyEntity(EntityID id) {
        entities.destroy(id);
        transforms.remove(id); velocities.remove(id);
        healths.remove(id); renderers.remove(id);
    }

    template<typename T> T& getComponent(EntityID id);
    template<typename T> void addComponent(EntityID id, T&& c);
    template<typename T> bool hasComponent(EntityID id);

    // Iteration sur les entites qui ont plusieurs composants
    template<typename... Ts>
    void forEach(std::function<void(EntityID, Ts&...)> fn) {
        // Trouve le storage le plus petit pour minimiser l'iteration
        auto& smallest = smallestStorage<Ts...>();
        for (size_t i = 0; i < smallest.size(); i++) {
            EntityID id = smallest.ids()[i];
            if ((hasComponent<Ts>(id) && ...)) // fold expression
                fn(id, getComponent<Ts>(id)...);
        }
    }

    // Acces direct aux storages pour l'iteration batch pure
    ComponentStorage<TransformComponent>& getTransforms() { return transforms; }
    ComponentStorage<RenderComponent>& getRenderers() { return renderers; }

private:
    EntityManager entities;
    ComponentStorage<TransformComponent> transforms;
    ComponentStorage<VelocityComponent> velocities;
    ComponentStorage<HealthComponent> healths;
    ComponentStorage<RenderComponent> renderers;
};
```

5. Systemes — exemples concrets moteur

```

// MovementSystem : iteration batch pure, acces direct au dense array
void movementSystem(World& w, float dt) {
    auto& transforms = w.getTransforms();
    auto& velocities = w.getVelocities();
    // Itere sur le storage le plus petit (entites avec velocity)
    for (size_t i = 0; i < velocities.size(); i++) {
        EntityID id = velocities.ids()[i];
        if (transforms.has(id))
            transforms.get(id).position += velocities.data()[i].linear * dt;
    }
}

// RenderSystem : collecte les draw calls, les envoie au renderer
void renderSystem(World& w, RHI& rhi) {
    auto& renderers = w.getRenderers();
    auto& transforms = w.getTransforms();
    for (size_t i = 0; i < renderers.size(); i++) {
        EntityID id = renderers.ids()[i];
        if (!transforms.has(id)) continue;
        rhi.submitDrawCall({
            renderers.data()[i].mesh,
            renderers.data()[i].material,
            transforms.get(id).position
        });
    }
}

// HealthSystem : detruit les entites mortes
void healthSystem(World& w) {
    std::vector<EntityID> toDestroy;
    auto& healths = w.getHealths();
    for (size_t i = 0; i < healths.size(); i++)
        if (healths.data()[i].current <= 0)
            toDestroy.push_back(healths.ids()[i]);
    // Detruire apres l'iteration pour ne pas invalider les iterateurs
    for (EntityID id : toDestroy) w.destroyEntity(id);
}

```

6. Ordre d'exécution des systemes

L'ordre des systemes est critique. Une bonne pratique : definir des phases d'exécution explicites.

- **Input phase** : InputSystem — lit les inputs, met a jour les velocites.
- **Update phase** : MovementSystem, AISystem, PhysicsSystem, HealthSystem.
- **Render phase** : CullingSystem, RenderSystem, UISystem.

Detruire les entites **toujours en fin de phase**, jamais pendant l'iteration d'un systeme.

7. Bonnes pratiques

- **Components = donnees pures** : jamais de logique dans un composant. Si tu ajoutes une methode `update()`, c'est un signe que la logique appartient a un systeme.
- **Ne pas detruire pendant l'iteration** : collecter les IDs a detruire, appliquer apres.
- **Acceder aux composants via le storage, pas via `forEach`** en hot path — l'iteration directe sur le dense array est toujours plus rapide.
- **Generation index** : toujours verifier `isAlive()` avant d'utiliser un ID stocke depuis une frame precedente.